

Контейнеры полезны не только в production. Их можно использовать и в средах разработки, и они предлагают несколько преимуществ. Та же проблема *works-on-my-machine* часто возникает, когда новый разработчик присоединяется к команде. Не говоря уже о головной боли от переключения версий runtime или локальной базы данных!

Например, [команда разработки программного обеспечения](#) в Хельсинкском университете имеет полностью [контейнеризованную среду разработки](#). Принцип во всех проектах разработки — иметь настройку, при которой новому разработчику нужно только установить Docker и клонировать код проекта с GitHub, чтобы начать работу. Ни одна зависимость никогда не устанавливается на хост-машину, Git, Docker и текстовый редактор по выбору — это единственное, что нужно.

Даже если ваше приложение не полностью контейнеризовано во время разработки, контейнеры могут быть очень полезны. Например, скажем, вам нужна MongoDB версии 4.0.22, установленная на порту 5656. Теперь это одна строка: "docker run -p 5656:27017 mongo:4.0.22" (MongoDB использует 27017 как порт по умолчанию).

Давайте контейнеризуем среду разработки NodeJS. Как вы, возможно, знаете, [NodeJS](#) — это кроссплатформенная среда выполнения JavaScript, которая позволяет запускать JavaScript на вашей машине, серверах и встроенных устройствах, среди многих других платформ.

Настройка требует некоторого понимания того, как работает NodeJS. Вот упрощенное объяснение, если вы не знакомы: библиотеки определяются в `package.json` и `package-lock.json` и устанавливаются с помощью `npm install`. [npm](#) — это менеджер пакетов Node.

Чтобы запустить приложение с пакетами, у нас есть скрипт, определенный в `package.json`, который инструктирует Node выполнить `index.js`, основной/входной файл в данном случае скрипт выполняется с помощью `npm start`. Приложение уже включает код для отслеживания изменений в файловой системе и перезапуска приложения, если обнаружены какие-либо изменения.

Проект "node-dev-env" находится здесь <https://github.com/docker-hy/material-applications/tree/main/node-dev-env>. Мы уже включили development Dockerfile и полезный docker-compose.yml.

Dockerfile

```
FROM node:16

WORKDIR /usr/src/app

COPY package* ./

RUN npm install
```

docker-compose.yml

```
version: '3.8'

services:
  node-dev-env:
    build: . # Build with the Dockerfile here
    command: npm start # Run npm start as the command
    ports:
      - 3000:3000 # The app uses port 3000 by default, publish it as 3000
```

```
volumes:
  - ./usr/src/app # Let us modify the contents of the container locally
  - node_modules:/usr/src/app/node_modules # A bit of node magic, this ensures
the dependencies built for the image are not available locally.
  container_name: node-dev-env # Container name for convenience

volumes: # This is required for the node_modules named volume
  node_modules:
```

И это все. Мы будем использовать том, чтобы скопировать весь исходный код внутрь тома, так что CMD будет запускать приложение, которое мы разрабатываем. Давайте попробуем!

```
$ docker compose up

...

Attaching to node-dev-env
node-dev-env |
node-dev-env | > dev-env@1.0.0 start
node-dev-env | > nodemon index.js
node-dev-env |
node-dev-env | [nodemon] 2.0.7
node-dev-env | [nodemon] to restart at any time, enter `rs`
node-dev-env | [nodemon] watching path(s): *.*
node-dev-env | [nodemon] watching extensions: js,mjs,json
node-dev-env | [nodemon] starting `node index.js`
node-dev-env | App listening in port 3000
```

Отлично! Начальный запуск немного медленный. Теперь, когда образ уже собран, это намного быстрее. Мы можем пересобрать всю среду всякий раз, когда захотим, с помощью `docker compose up --build`.

Давайте посмотрим, работает ли приложение. Используйте браузер для доступа к <http://localhost:3000>, он должен выполнять простое сложение с query params.

Однако вычисление не имеет смысла! Давайте исправим ошибку. Бьюсь об заклад, это эта строка прямо здесь <https://github.com/docker-hy/material-applications/blob/main/node-dev-env/index.js#L5>

Когда я меняю строку, на моей хост-машине приложение мгновенно замечает, что файлы изменились:

```
$ docker compose up

...

Attaching to node-dev-env
node-dev-env |
node-dev-env | > dev-env@1.0.0 start
node-dev-env | > nodemon index.js
node-dev-env |
node-dev-env | [nodemon] 2.0.7
node-dev-env | [nodemon] to restart at any time, enter `rs`
node-dev-env | [nodemon] watching path(s): *.*
node-dev-env | [nodemon] watching extensions: js,mjs,json
node-dev-env | [nodemon] starting `node index.js`
node-dev-env | App listening in port 3000
```

```
node-dev-env | [nodemon] watching extensions: js,mjs,json
node-dev-env | [nodemon] starting `node index.js`
node-dev-env | App listening in port 3000
node-dev-env | [nodemon] restarting due to changes...
node-dev-env | [nodemon] starting `node index.js`
node-dev-env | App listening in port 3000
```

И теперь обновление страницы показывает, что наше изменение кода исправило проблему. Среда разработки работает.

Следующее упражнение может быть чрезвычайно легким или чрезвычайно сложным. Не стесняйтесь веселиться с ним.

Упражнение 2.11

:::info Упражнение 2.11

Выберите некоторые из ваших собственных проектов разработки и начните использовать контейнеры в среде разработки.

Объясните, что вы сделали. Это может быть что угодно, например, поддержка `docker-compose.yml` для контейнеризации сервисов (таких как базы данных) или даже полностью контейнеризованная среда разработки.

:::

Если вас интересует, как создать контейнеризованную среду разработки для React/Node Single page web app, пожалуйста, посмотрите на курс [Full stack open](#), который имеет одну [главу](#), посвященную этой теме.